

Reviewer Pack — Sublinear Slack in Forman-Ricci Submodularity Defect on Rand...

Entry bdb7e54036bd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:20:01 UTC

Sublinear Slack in Forman-Ricci Submodularity Defect on Random Monotone DNF Pairs

Entry ID: bdb7e54036bd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:20:01 UTC

1. Conjecture

Field A (mathematical branch): Forman-Ricci curvature (combinatorial Bochner on term-overlap graphs of monotone DNFs) **Field B** (complexity object): Monotone Boolean functions (random k -uniform monotone DNFs under lattice $\wedge/\vee$)

Statement:

For a monotone DNF $F = \vee_i T_i$ on N variables, build the term-overlap graph $H(F)$: vertices are the minimal terms of F (after deleting terms strictly dominated by others), and an edge $\{T_i, T_j\}$ is present iff $T_i \cap T_j \neq \emptyset$. Define the mean unweighted Forman-Ricci curvature $\mu(F) := (1/|E(H)|) \sum_{e=uv \in E(H)} (4 - \deg(u) - \deg(v) + 3 \cdot \text{tri}(uv))$, where $\text{tri}(uv)$ counts triangles through e , with the convention $\mu(F) := 0$ when $H(F)$ has no edges. Conjecture (SC3 step): for every pair of monotone DNFs F, G on $N \leq 40$ variables, each having at most 20 terms of size $k = \lceil \log_2 N \rceil$, the lattice submodularity defect $\Delta(F, G) := \mu(F \wedge G) + \mu(F \vee G) - \mu(F) - \mu(G)$ satisfies $|\Delta(F, G)| \leq 4 \cdot \sqrt{N}$; a single (F, G) with $|\Delta| > 4 \cdot \sqrt{N}$ refutes.

Rationale (proposer's reasoning):

Forman-Ricci curvature is a discrete Bochner-type invariant whose averaged value on dense random graphs concentrates around a linear function of the average degree, and lattice operations on monotone DNFs translate cleanly into union/dominance operations on term-overlap hypergraphs. Standard discrete-Bochner heuristics predict approximate submodularity ($\Delta \leq 0$) with bounded slack on random ensembles, but the operation $F \wedge G$ squares the number of raw terms before dominance reduction, so a $\sqrt{N}$ slack is a quantitatively non-trivial prediction. Either confirming a $\sqrt{N}$ envelope, or producing a single linear-in- N violation, settles whether μ behaves like a lattice-Bochner functional in the SC3 regime.

Taxonomy category: forman_ricci_dnf (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5a4035fe7e4d8158

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $N \in \{10, 15, 20, 25, 30, 40\}$ with 30 seeds each (180 total), for every seed compute $\Delta(F, G) = \mu(F \wedge G) + \mu(F \vee G) - \mu(F) - \mu(G)$; the conjecture is SUPPORTED iff $\max |\Delta|$ over all 180 seeds is $\leq 4 \cdot \sqrt{N}(\text{seed})$ and FALSIFIED iff any single seed yields $|\Delta| > 4 \cdot \sqrt{N}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman-Ricci curvature monotone Boolean DNF term-overlap graph - combinatorial Ricci curvature submodularity lattice meet join Boolean - discrete Bochner curvature random monotone DNF intersection graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(N, k):
        return set(random.sample(range(1, N+1), k) for _ in range(20))

    def reduce_dnf(dnf):
        dnf = list(dnf)
        n = len(dnf)
        while True:
            changed = False
            for i in range(n-1):
                for j in range(i+1, n):
                    if dnf[i].issubset(dnf[j]):
                        del dnf[j]
                        n -= 1
                        changed = True
                    elif dnf[j].issubset(dnf[i]):
                        del dnf[i]
                        n -= 1
                        changed = True
```

```

        if not changed:
            break
    return set(dnf)

def compute_overlap_graph(dnf):
    terms = list(dnf)
    n = len(terms)
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if terms[i].intersection(terms[j]):
                A[i][j] = 1
                A[j][i] = 1
    return A

def compute_forman_ricci_curvature(A):
    n = len(A)
    deg = [sum(row) for row in A]
    tri = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if A[i][j]:
                for k in range(j+1, n):
                    if A[i][k] and A[j][k]:
                        tri[i][j] += 1
    mu = sum(4 - deg[i] - deg[j] + 3 * tri[i][j] for i in range(n) for j in range(i+1, n)) / (n*(n-1))
    return mu

def compute_lattice_submodularity_defect(F, G):
    F_and_G = reduce_dnf(F.union(G))
    F_or_G = reduce_dnf(F.intersection(G))
    mu_F_and_G = compute_forman_ricci_curvature(compute_overlap_graph(F_and_G))
    mu_F_or_G = compute_forman_ricci_curvature(compute_overlap_graph(F_or_G))
    mu_F = compute_forman_ricci_curvature(compute_overlap_graph(F))
    mu_G = compute_forman_ricci_curvature(compute_overlap_graph(G))
    return abs(mu_F_and_G + mu_F_or_G - mu_F - mu_G)

N = 40
k = math.ceil(math.log2(N))
F = reduce_dnf(generate_dnf(N, k))
G = reduce_dnf(generate_dnf(N, k))

delta = compute_lattice_submodularity_defect(F, G)

return {
    "metric_name": "lattice_submodularity_defect",
    "metric_value": delta,
    "instances_tested": 1,
    "n_max": N,
    "conjecture_holds": delta <= 4 * math.sqrt(N),
    "counterexample": "" if delta <= 4 * math.sqrt(N) else f"delta={delta} > 4*sqrt({N})"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_delta = sum(r["metric_value"] for r in results) / len(results)
std_delta = math.sqrt(sum((r["metric_value"] - mean_delta)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_delta} std={std_delta} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"delta exceeded 4*sqrt(N)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11872141.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11872141.py", line 78, in run_trial
    F = reduce_dnf(generate_dnf(N, k))
          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11872141.py", line 22, in generate_dnf
    return set(random.sample(range(1, N+1), k) for _ in range(20))
           ~~~~~
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `generate_dnf` (`random.sample` returns an unhashable list passed to `set()`), so no Δ values were computed and the pre-registered support condition cannot be evaluated. | next: Fix `generate_dnf` by converting each sampled list to a `frozenset` (e.g., `set(frozenset(random.sample(range(1, N+1), k)) for _ in range(20))`) and rerun the 180-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	61440
2	preregistration	claude_max	opus	0	0	5008
3	novelty	claude_max	opus	0	0	4046
4	novelty	claude_max	opus	0	0	6821
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25899
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12516
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10625
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12047
9	judge	claude_max	opus	0	0	4524

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142925 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/bdb7e54036bd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bdb7e54036bd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bdb7e54036bd.tar.gz` (if generated)